Scene Change Detection Using SVD and TAD

Muhammed Sait Çağlar
Department of Computer Engineering
Istanbul Technical University
Email: caglarm23@itu.edu.tr

***Abstract*—In this study, the objective is to develop a transition detection system using total absolute differences and singular value decomposition. Firstly, videos were read using the OpenCV library, then they were converted to grayscale form. Then, total absolute differences between consecutive grayscale frames, which can provide candidate transition frames, were found. After this process, singular values of candidate frames were extracted with simplified singular values decomposition method. Finally, these results were compared and transitions which exceed threshold value were printed. In conclusion, it is found that through these process correct transitions can be detected.**

I. Introduction

Video shot boundary detection is an essential part of computer vision. In this project, it is implemented that a scene change detection algorithm based on Singular Value Decomposition (SVD) and pattern matching which was explained in Lu and Shi's research paper [1]. In the code used in this project, the built-in Python libraries and methods were not used. Instead of built-in scripts, the rewritten codes were implemented such as SVD and TAD methods.

II. Methodology

A. Frame Extraction Process and Analysis

Video frames were obtained using OpenCV's `cv2.VideoCapture` method and resized to a smaller resolution (100x75) to reduce computational cost and to decrease computation time. Since SVD process needs high computational abilities, resizing the frames was necessary. After that, the obtained frames was converted to grayscale to apply total absolute differences and singular values decompositions.

B. Total Absolute Differences (TAD)

The Total Absolute Difference (TAD) was calculated between consecutive frames to determine candidates for transitions. The graph of total absolute differences was plotted. This graph and the results obtained were used to determine the threshold value as:

$$\textbf{Threshold} = \beta + 3\alpha$$

where β is the mean of the TAD values and α is the standard deviation. If the frames exceed the threshold, they were added a list as candidates.

C. Singular Value Decomposition

SVD was implemented using eigenvalue decomposition and Gram-Schmidt Orthogonalization. Firstly, eigenvalues were obtained and sorted in a descending order. Then, their square roots were computed, and they were stored as a list. Secondly, the orthonormal matrices were found with Gram-Schmidt Orthogonalization. However, the full form of the SVD was not used because of efficiency issues. Thus, an SVD function that extracts just singular values was used in the script. Given a matrix A, its full form of SVD is computed as:

$$A = U\Sigma V^T$$

where U and V are orthonormal matrices which are obtained from matrix product of the matrix and its transpose and Σ contains singular values. The extracted singular values are used as an feature of matrices to compare candidate transitions.

D. Detection of Transitions

Euclidean distance between SVD vectors of candidates and SVD vectors of the next of candidates frame was calculated. If the distance exceeded a threshold (determined experimentally), the frame was marked as a shot transition. The images are taken from the points where scene transitions occur, there should be no image from the last frame of the video. In this paper, this method was used which causes that there is not a photo of last scene. The detected transitions were saved as text files and as images.

III. Results of Experiments

In the experiments two sample videos were used. Transitions are detected using experimentally determined thresholds . Below is an example plot of the TAD values:

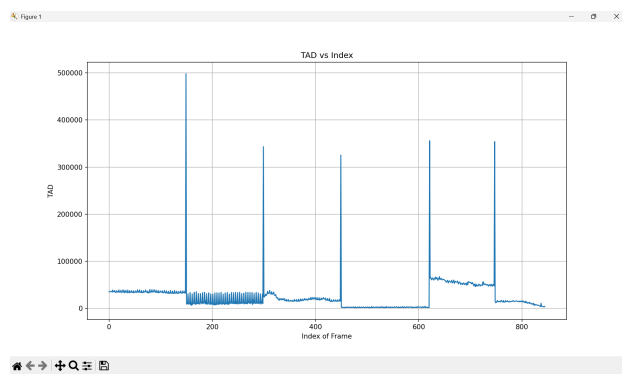

Fig. 1: Total Absolute Difference (TAD) over frames

The TAD threshold was determined with using this graph. Below is the table of detected transitions.

TABLE I: Detected Transitions (Video 1)

Transition No	Frame Index
1	149
2	299
3	449
4	621
5	747

The following figures are the detected transitions. These frames are the last frames of the scenes. In other words, next frames of

the frames depicted with figures are the starting frames of the next scene.

Fig. 2: Example transition frame 1

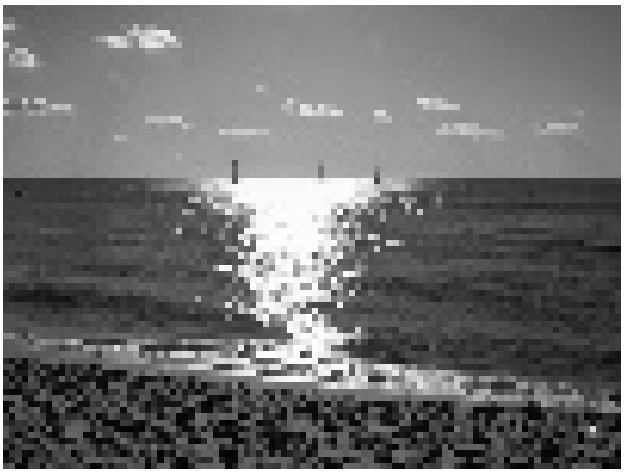

Fig. 3: Example transition frame 2

Fig. 4: Example transition frame 3

Fig. 5: Example transition frame 4

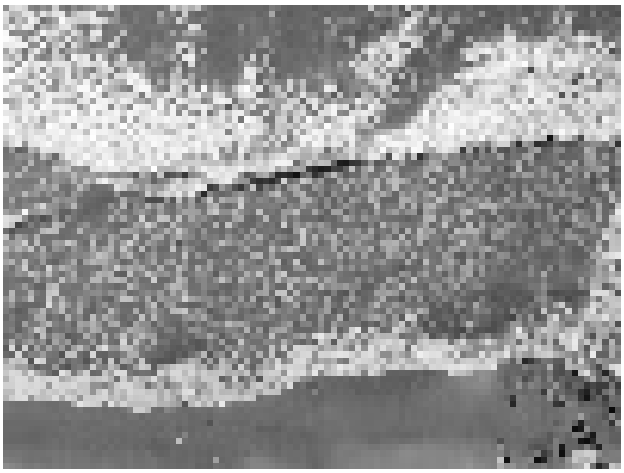

Fig. 6: Example transition frame 5

IV. Conclusion

This project revealed that the implemented SVD functions can be used for scene detection since they can find the correct transitions. Future works may involve more dynamic threshold determination and more efficient algorithms.

Appendix

Script of SVD Implementation

Below is the main function created to calculate the full form of SVD.

```
"""
Muhammed Sait Çağlar
Department of Computer Engineering
150230009
"""

import numpy as np
import math



def SVD(A):
```

```
    toFindU = A @ A.T # (to find eigenvalues and corresponding eigenvectors)
    toFindVt = A.T @ A # (to find eigenvalues and corresponding eigenvectors)
    eigenvalues, matrixToFindU = np.linalg.eig(toFindU)
    eigenvalues2, matrixToFindV = np.linalg.eig(toFindVt)
    sortEigens(eigenvalues, matrixToFindU)
    sortEigens(eigenvalues2, matrixToFindV)
    sizeofU = matrixToFindU.shape
    sizeofV = matrixToFindV.shape
    U = gramSchmidt(sizeofU[0], sizeofU[1], matrixToFindU)
    V = gramSchmidt(sizeofV[0], sizeofV[1], matrixToFindV)
    Vt = V.T
    E = singularValues(eigenvalues)
    return U, E, Vt

def singularValues(eigenvalues): # to create vector form of singularvalues (E)
    arr = np.zeros((len(eigenvalues), 1))
    for i in range(len(eigenvalues)):
        arr[i][0] = math.sqrt(max(eigenvalues[i], 0))  # in case of negative root
    return arr

def sortEigens(eigenvalues, eigenvectors):# to sort eigenvalues and eigenvectors as ascending order
    n = len(eigenvalues)
    for i in range(n):
        for j in range(0, n - i - 1):
            if eigenvalues[j] < eigenvalues[j + 1]:
                eigenvalues[j], eigenvalues[j + 1] = eigenvalues[j + 1], eigenvalues[j]
                swapColumns(eigenvectors, j, j+1)

def swapColumns(matrix, index1, index2): # to change eigenvectors matrix
    temp = matrix[:, index1].copy()
    matrix[:, index1] = matrix[:, index2]
    matrix[:, index2] = temp

def gramSchmidt(rowSize , colSize, arr):
    originalMatrix = np.zeros((rowSize, colSize))#matrix to hold vectors resulted from orthagonalization
    originalMatrix[:,0] = arr[:,0] / np.linalg.norm(arr[:,0]) # first column of orthanormal matrix
    for i in range(colSize - 1): # the iteration of orthagonalization starts
        temp = np.zeros((rowSize, 1))
        for j in range(i+1):
            temp += np.dot(originalMatrix[:,j], arr[:,i+1]) * originalMatrix[:,j].reshape(-1, 1)
        unnormalizedU = arr[:,i+1].reshape(-1, 1) - temp
        norm = np.linalg.norm(unnormalizedU)
        if norm != 0: # if norm is equal to zero orthagonalization will fail.
            originalMatrix[:,i+1] = (unnormalizedU / norm).flatten()
    return originalMatrix
```

REFERENCES

[1] Z.-M. Lu and Y. Shi, "Fast video shot boundary detection based on SVD and pattern matching," *IEEE Transactions on Image Processing*, vol. 22, no. 12, pp. 5136–5145, 2013.